EXPERIMENTNO.

MONGODB-CREATE USERS AND ROLES

1)Write a MongoDB query to creates a user named reportsUser in the admin database but does not yet assign roles:

- **usestudent**
- **db.createUser({ user: "reportsUser", pwd: passwordPrompt(), roles: [ ] }**

2. Write a MongoDB query to creates a user named appAdmin in the student database and gives the user readWrite access to the config database.

- **db.createUser({ user: "appAdmin",pwd: passwordPrompt(), roles:[{ role: "readWrite", db: "config" }]})**

Output { ok: 1 }

3. Write a MongoDB query to creates a user named restricted in the student database. This user may only authenticate if connecting from IP address 192.0.2.0 to IP address 198.51.100.0

- **db.createUser({user: "restricted", pwd: passwordPrompt(), roles: [{ role: "readWrite", db: "reporting" }],authenticationRestrictions: [{clientSource: ["192.0.2.0"], serverAddress: ["198.51.100.0"]}]})**

4. Write a MongoDB query to drops the reportUser user on the student database.

- **db.dropUser("reportsUser", {w: "majority", wtimeout: 5000})**

5)Write a MongoDB query to operations return information about an example appAdmin user in an student database.

● **db.getUser("appAdmin") { _id: 'admin.appAdmin', userId: UUID("60cfa9b0-d876 44e4-be1f-928a75532865"), user: 'appAdmin', db: 'admin', roles: [{ role: 'clusterAdmin', db: 'admin' }, { role: 'readWrite', db: 'config' }], mechanisms: ['SCRAM-SHA-1', 'SCRAM-SHA-256'] }**

5)Write a MongoDBquery to gives appAdmin the readWrite role on the products database and the read role on the student database.

● **db.grantRolesToUser("appAdmin",["readWrite" , { role: "read", db: "student" }],{ w: "majority" , wtimeout: 4000 })**

6)Write a MongoDB query to method removes the user's roles: the read role and the readWrite role on the student database.

● **db.revokeRolesFromUser("appAdmin",[{ role: "read", db: "student" }, "readWrite"],{ w: "majority" })**